UNIT - III

Divide and Conquer Method-Introduction-Binary Search-Finding Min Max-Maximum Subarray Problem-Towers of Hanoi Problem-Finding the kth element-Analysis of Quick and Merge Sort.

DIVIDE AND CONQUER METHODOLOGY

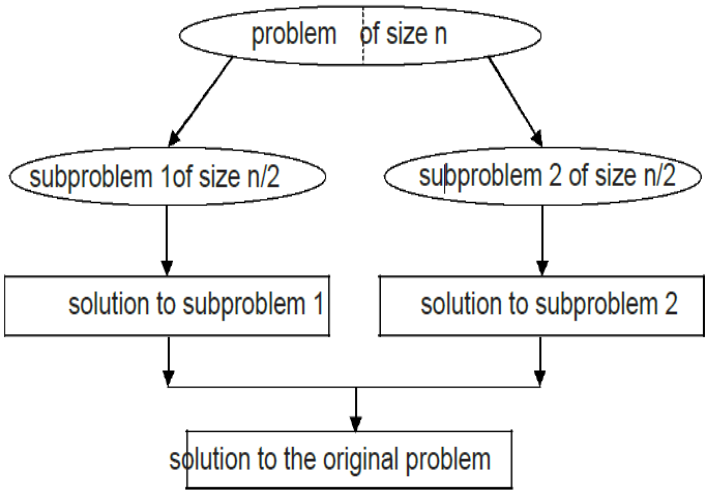
A divide and conquer algorithm work by recursively breaking down a problem into two or more sub-problems of the same (or related) type (divide), until these become simple enough to be solved directly (conquer).

Divide-and-conquer algorithms work according to the following general plan:

1. A problem is divided into several subproblems of the same type, ideally of about equal size.

2. The subproblems are solved (typically recursively, though sometimes a different algorithm is employed, especially when subproblems become small enough).

3. If necessary, the solutions to the subproblems are combined to get a solution to the original problem.



The divide-and-conquer technique as shown above, depicts the case of dividing a problem into two smaller subproblems, then the subproblems solved separately. Finally, solution to the original problem is done by combining the solutions of subproblems.

1.BINARY SEARCH

A binary search is efficient algorithm to find the position of a target (key) value within a sorted array.

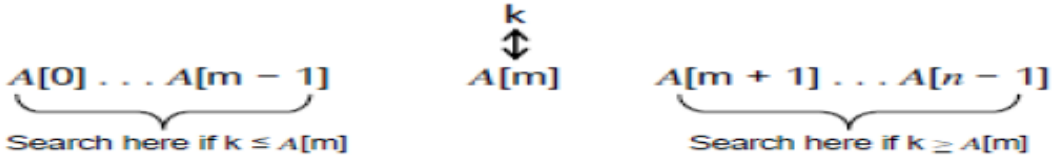
 The binary search algorithm begins by comparing the target value to the value of the middle element of the sorted array. If the target value is equal to the middle element's value, then the position is returned and the search is finished.

 If the target value is less than the middle element's value, then the search continues on the lower half of the array.

 If the target value is greater than the middle element's value, then the search continues on the upper half of the array.

 This process continues, eliminating half of the elements, and comparing the target value to the value of the middle element of the remaining elements - until the target value is either found (position is returned).

Binary search is a remarkably efficient algorithm for searching in a sorted array (Say A). It **works by comparing a search key K with the array's middle element A[m]. If they match, the** algorithm stops; otherwise, the same operation is repeated recursively for the first half of the array if K <A[m], and for the second half if K >A[m]:



Though binary search is clearly based on a recursive idea, it can be easily implemented as a non-recursive algorithm, too. Here is pseudocode of this non-recursive version.

ALGORITHM *BinarySearch*(*A*[0..*n* − 1], *K*)

//Implements nonrecursive binary search

//Input: An array *A*[0..*n* − 1] sorted in ascending order and a search key *K*

//**Output: An index of the array's element that is equal to** *K*/ or −1 if there is no such element

l ← 0; *r* ← *n* − 1

+ r /

while *l* ≤ *r* **do**

m ←

if *K* = *A*[*m*] **return** *m*

else if*K* <*A*[*m*]

r ← *m* − 1

else *l* ← *m* + 1

return −1

The standard way to analyze the efficiency of binary search is to count the number of times the search key is compared with an element of the array (three-way comparisons). One comparison of K with A[m], the algorithm can determine whether K is smaller, equal to, or larger than A[m].

As an example, let us apply binary search to searching for $K = 70$ in the array. The iterations of the algorithm are given in the following table:

index	0	1	2	3	4	5	6	7	8	9	10	11	12
value	3	14	27	31	39	42	55	70	74	81	85	93	98
iteration 1	*l*						*m*						*r*
iteration 2								*l*		*m*			*r*
iteration 3								*l,m*	*r*				

The worst-case inputs include all arrays that do not contain a given search key, as well as some successful searches. Since after one comparison the algorithm faces the same situation but for an array half the size,

The number of key comparisons in the worst case $C_{worst}(n)$ by recurrence relation.

$C_{worst}(n) = C_{worst}\left(\left\lfloor\frac{n}{2}\right\rfloor\right) + 1 \; for \; n > 1, C_{worst}(1) = 1.$

$\therefore C_{worst}(n) = \lfloor log_2 n \rfloor + 1 = \lceil log_2(n+1) \rceil \quad \because C_{worst}(2^k) = (k+1) = log_2 k + 1$ for $n=2^k$

- First, The worst-**case time efficiency of binary search is in Θ(log n).**
- Second, the algorithm simply reduces the size of the remaining array by half on each
- iteration, the number of such iterations needed to reduce the initial size *n* to the final size 1 has to be about log2 *n*.
- Third, the logarithmic function grows so slowly that its values remain small even for very large values of *n*.
- The average case slightly smaller than that in the worst case $Cavg(n) \approx \log 2\ n$
- The average number of comparisons in a successful is $\boldsymbol{Cavg(n) \approx \log 2\ n - 1}$
- The average number of comparisons in an unsuccessful is $\boldsymbol{Cavg(n) \approx \log 2(n+1)}$.

(ii)Maximum Sum Subarray Problem

Maximum Subarray Sum using Divide and Conquer algorithm:

* Give a one dimensional array that may contain positive and negative integers.
* Find the sum of contiguous subarray of numbers which has the largest sum.
* For ex: if the given array is {-2, -5, 6, -2, -3, 1, 5, 6} then the maximum subarray sum is 7.

The naive Method:

* It is to run two-loops
* The outer loop picks the beginning element, the inner loop finds the maximum possible sum with first element picked by outer loop and compares the maximum with the overall maximum.
* Finally, return the overall maximum.
* The time complexity of the Naive method is $O(n^2)$.

Divide and Conquer Approach:

* Can find the maximum subarray sum in $O(n \log n)$ time.
* Following is the Divide and conquer algorithm
1. Divide the given array in two halves.
2. Return the maximum of following three
   a. Maximum subarray sum in left half (Make a recursive call)
   b. " " right half (Make a recursive call)
   c. " " such that subarray crosses the midpoint
3. The lines 2.a and 2.b are simple recursive calls.

How to find maximum subarray sum such that the subarray crosses the midpoint?

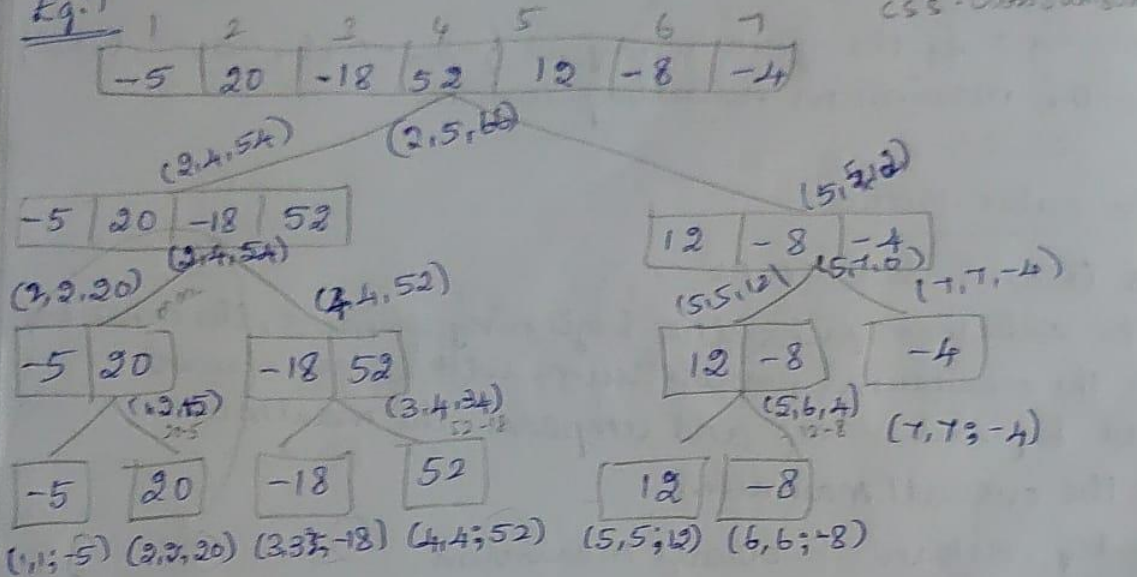
* We can easily find the crossing sum in linear time
* Find the maximum sum starting from mid point and ending at some point on left of mid

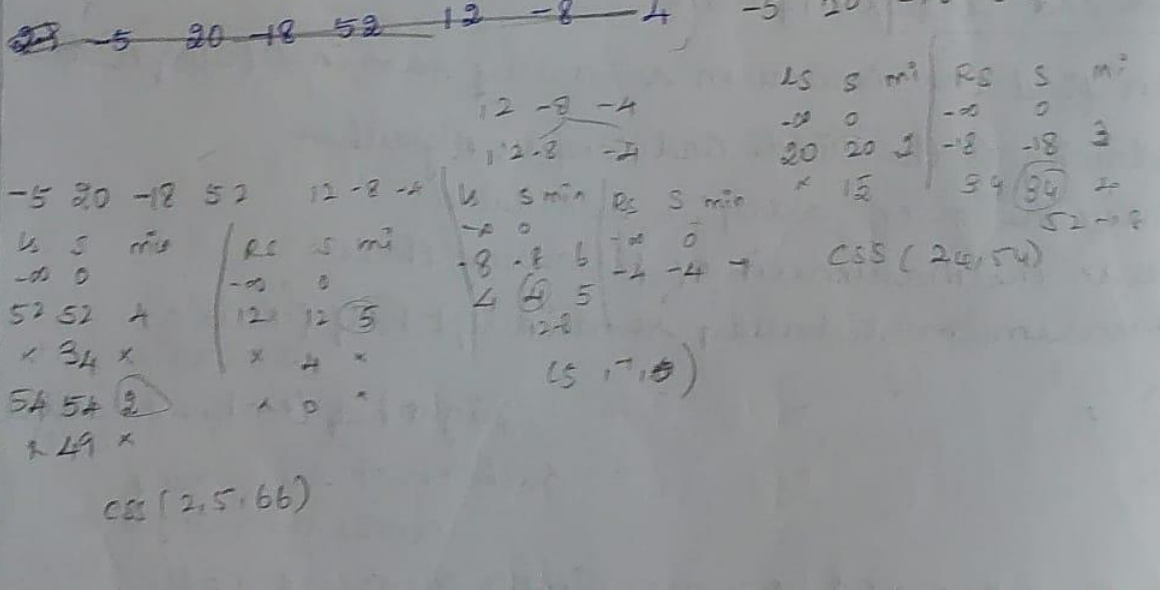
*Then find the maximum sum starting from mid+1 and ending with some point of right of mid+1.

*Finally, combine the two and return the maximum among left, right and combination of both.

LSS - Left
RSS - Right
CSS - Cross sum

Eg:1





Find the kth smallest element in an unsorted array

The Divide and Conquer approach can be used to find the **kth smallest element** in an unsorted array efficiently. This method is commonly known as the **QuickSelect algorithm**. It is similar to the QuickSort algorithm but instead of sorting the entire array, it partially sorts the array to find the kth smallest element.

Here's the basic idea:

1. **Choose a Pivot**: Like QuickSort, pick a pivot element from the array.
2. **Partition the Array**: Partition the array into two parts: elements less than the pivot and elements greater than or equal to the pivot.
3. **Recur in One Half**:

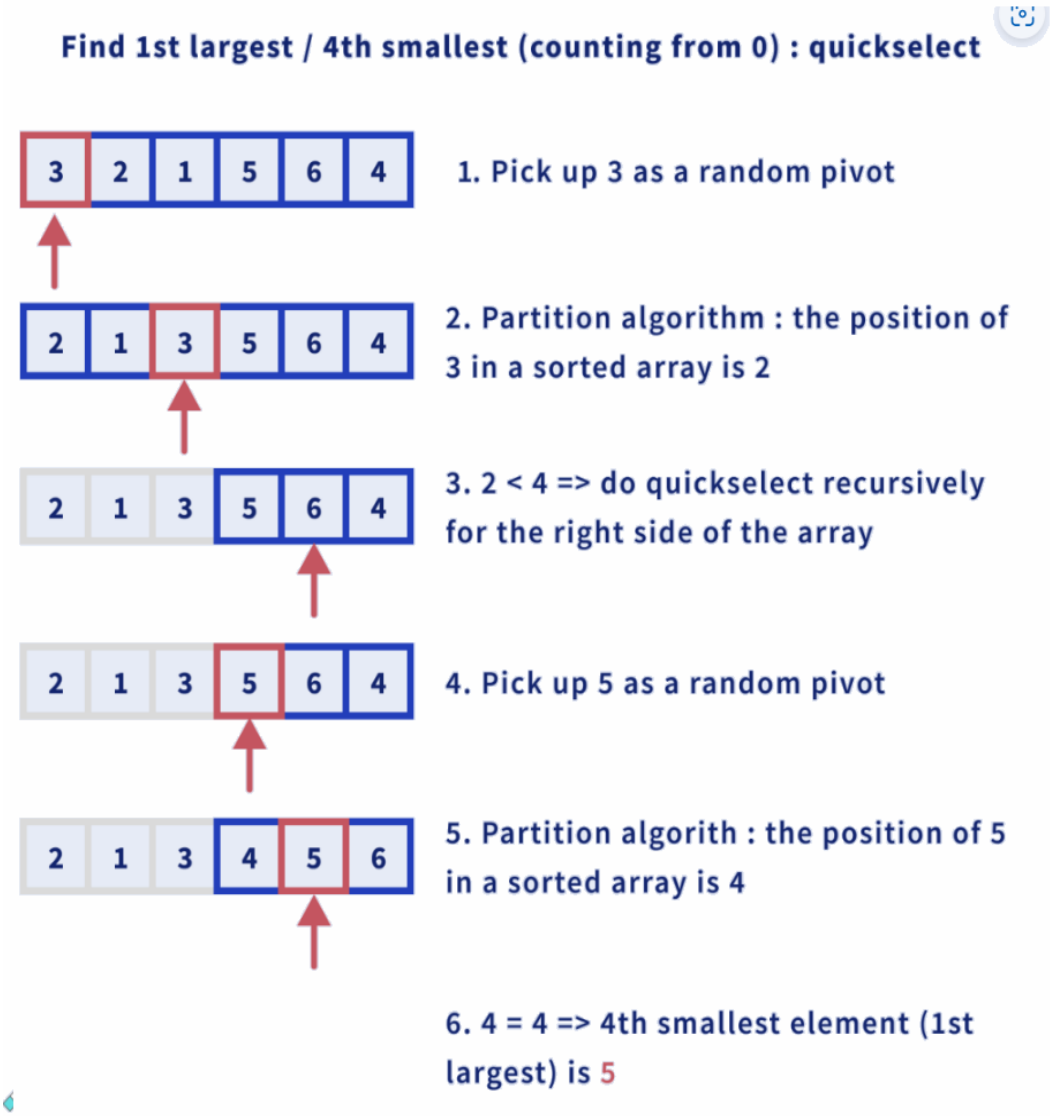
    a. If p == k, the pivot is the **kth smallest element**.
    b. If p > k, the kth smallest element lies in the left half (since all smaller elements are there). We recur only on the left part of the array.
    c. If p < k, the kth smallest element lies in the right half (since the required element is greater than the pivot). We recur on the right part of the array.

The QuickSelect algorithm has an average time complexity of **O(n)**, although the worst case can be **$O(n^2)$** if poor pivot choices are made.

Algorithm:

```
Algorithm  quickselect(arr, low, high, k)
{
   if (low <= high) then
      pi = partition(arr, low, high)
      if (pi == k) then
         return arr[pi]
      else if ( pi > k)
         return quickselect(arr, low, pi - 1, k)
      else:
         return quickselect(arr, pi + 1, high, k)
   }
Algorithm  partition(arr, low, high){
{
   pivot = arr[low]
   i = low , j = high + 1
repeat
         repeat i  = i+1  until arr[i] >= pivot
         repeat j = j -1 until arr[j] <= pivot
         swap(arr[i],arr[j])
until I <= j
swap(arr[i],arr[j])
return j
}
```

Example:


Find 1st largest / 4th smallest (counting from 0) : quickselect
3 2 1 5 6 4
1. Pick up 3 as a random pivot
2 1 3 5 6 4
2. Partition algorithm : the position of 3 in a sorted array is 2
2 1 3 5 6 4
3. 2 < 4 => do quickselect recursively for the right side of the array
2 1 3 5 6 4
4. Pick up 5 as a random pivot
2 1 3 4 5 6
5. Partition algorith : the position of 5 in a sorted array is 4
6. 4 = 4 => 4th smallest element (1st largest) is 5

FINDING MINIMUM AND MAXIMUM.

PROBLEM STATEMENT:

The problem is to find the minimum and maximum element from a given set of n elements.

BRUTE - FORCE METHOD [STRAIGHT FORWARD APPROACH]:

Algorithm:

```
void StraightMaxMin(Type a[], int n, Type &max, Type &min)
{
    max = min = a[1]
    for (int i=2; i<=n; i++)
    {
        if (a[i]->max)  ---------> (n-1)
            count++
            max = a[i]

        if (a[i] < min)  ---------> (n-1)
            count++
            min = a[i]

    }
}
```

ANALYSIS:

(i) Input size - n

(ii) Basic Operation - Comparison.

Scanned by CamScanner

Scanned by CamScanner

(iii) Time Complexity:

$$T(n) = 2(n-1)$$

IMPROVEMENT IN BRUTE-FORCE METHOD:

An immediate improvement is possible by realizing that the comparison $a[i] < min$ is necessary only when $a[i] > max$ becomes false. Thus, the code can be re-written as,

```
:
if(a[i] > max)
    max = a[i]
else if (a[i] < min)
    min = a[i]
```

BEST-CASE & WORST-CASE:

→ Best-Case occurs when the elements are in increasing order. Then the no. of comparisons is $(n-1)$

→ Worst-Case occurs when the elements are in decreasing order. Then the no. of comparisons is $2(n-1)$

Scanned by CamScanner

Scanned by CamScanner

Solving Minimum Maximum Using Divide & Conquer

Strategy:

Lets have $P = (n, a[i] \ldots a[j]$ denoting an arbitrary instance of a problem. Here n is the number of elements in the list $a[i] \ldots a[j]$. Goal is to find minimum and maximum element in the given list. There are three possibilities,

(a) if $n = 1$, i.e., only one element in the list,

then, minimum = maximum = $a[i]$

(b) if $n = 2$, i.e., two elements are in the list,

then the problem is solved using one comparison,

i.e., if $a[i] > a[2]$ then minimum = $a[2]$; maximum = $a[1]$

else minimum = $a[i]$, maximum = $a[2]$.

(c) if $n > 2$, then the list is divided into two instances,

$$P_1 = (a[1] \ldots a[n/2]) \text{ and } P_2 = (a[n/2]+1 \ldots a[n]).$$

After dividing P into two smaller sub-problems, we can solve them by recursively invoking the same divide and conquer strategy to the sub-problems.

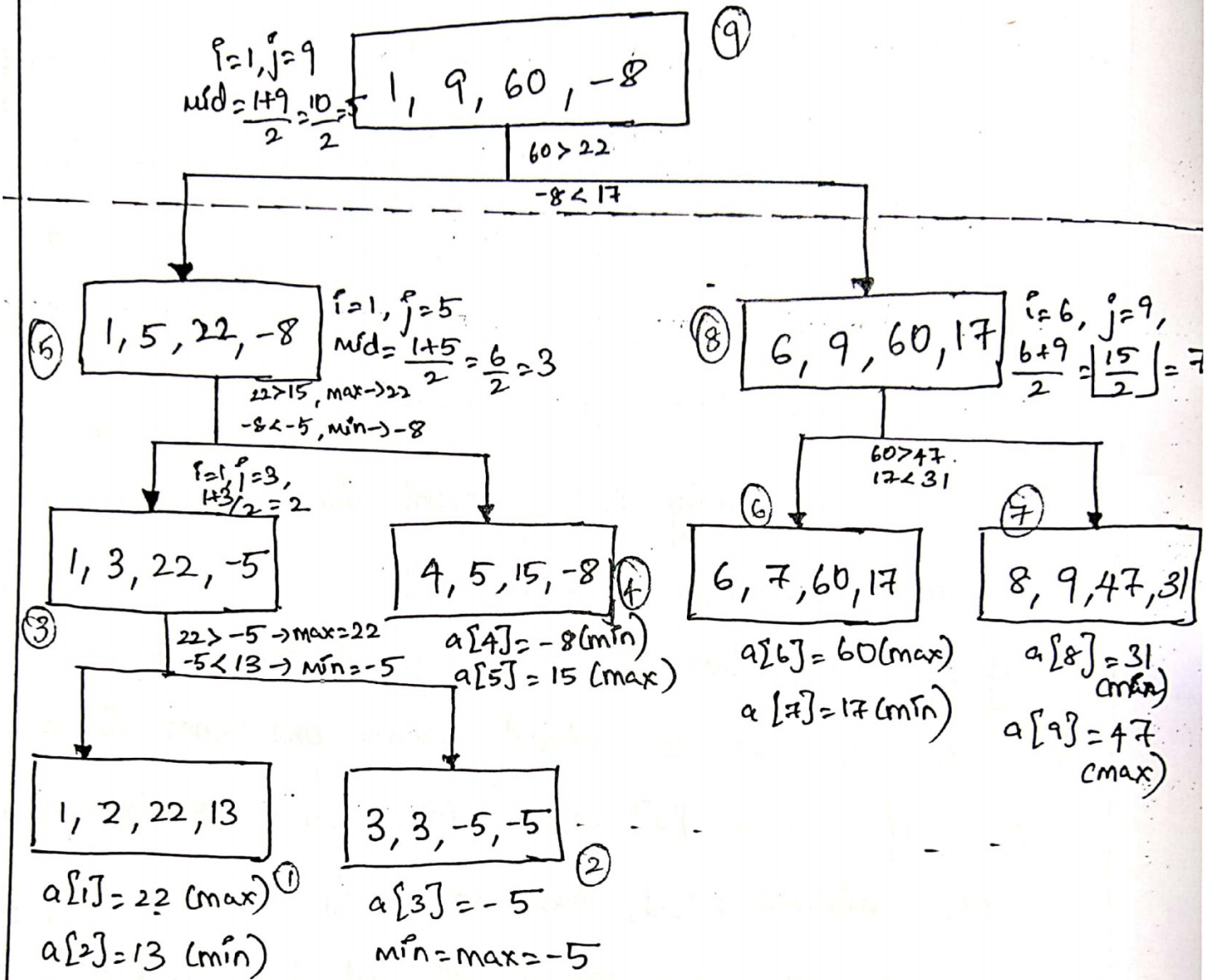
Scanned by CamScanner

Scanned by CamScanner

a : 22 13 -5 -8 15 60 17 31 47

index : 0 1 2 3 4 5 6 7 8 9

i



* Initially, nodes are drawn, with i and j value which starts from root node [1,9,--] to leaf [3,3,--]

* Then the circled number of each node represents the order in which min and max value are calculated.

Scanned by CamScanner

Scanned by CamScanner

Algorithm:

```
void MinMax(int i, int j, Type &max, Type &min)
{ // a[1...n] is a global array. Parameters i and j are
  // integers, 1<=i<=j<=n.
{
  if (i==j)                          //only one element in
                                     the array.
     max = min = a[i]
  else if (i== j-1)                  // if two elements are
                                        present in the array.
  {
     if (a[i] < a[j])
     {
          min = a[i]
          max = a[j]
     }
     else
     {
        max = a[i]
        min = a[j]
     }
  }
  else                               // if n > 2 elements.
  {
     int mid = ⌊(i+j)/2⌋
     MinMax(i, mid, max, min)
     MinMax(mid+1, j, max1, min1)
     if (max < max1)  max = max1
     if (min > min1)  min = min1
  }
}
```

Scanned by CamScanner

Scanned by CamScanner

TIME COMPLEXITY:

Recurrance Relation for the given construct is,

$$T(n) = \begin{cases} T(n/2) + T(n/2) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

So, $\therefore$, $T(2) = 1$

$$T(n) = 2T(n/2) + 2 \longrightarrow (1)$$

Put $n = n/2$

$$T(n/2) = 2T(n/4) + 2 \longrightarrow (2)$$

Put (2) in (1)

$$T(n) = 2\left[2T(n/4) + 2\right] + 2$$

$$T(n) = 4T(n/4) + 4 + 2 \longrightarrow (3)$$

Find $T(n/4)$, put $n = n/4$ in (1)

$$T(n/4) = 2T(n/4/2) + 2$$

$$T(n/4) = 2T(n/8) + 2 \longrightarrow (4)$$

Scanned by CamScanner

Scanned by CamScanner

Put (4) in (3)

$$T(n) = 4\left[2T\left(n/8\right)+2\right]+4+2$$

$$T(n) = 8T\left(n/8\right)+8+4+2$$

~~Expressing in terms of i,~~

$$T(n) = 2^{i-1}T\left(n/2^{i-1}\right) + 2^{i} - 2 \longrightarrow (5)$$

Equate (5) in Initial condition, $[T(2)]$.

Put $n = 2^i$

$$T(n) = 2^{i-1}T\left(\frac{2^i}{2^{i-1}}\right) + 2^i - 2$$

$$= 2^{i-1}T\left(\frac{n}{n/2}\right) + 2^i - 2$$

$$= \frac{n}{2}T(2) + n - 2$$

$$= \frac{n}{2} + n - 2$$

$$T(n) = \frac{3n}{2} - 2$$

$n = 2^i$

$2^{i-1} = 2^i \times 2^{-1}$

$= 2^i \times \frac{1}{2}$.

$$2^{i-1} = \frac{n}{2}$$

Scanned by CamScanner

Scanned by CamScanner

Comparing Straightforward Method And Divide And Conquer Method:

Straight forward Method (Brute-Force) = $2(n-1)$ comparisons.

Divide & Conques Method = $\frac{3n}{2} - 2$ comparisons.

Divide and Conques saves 25% of comparisons.

$$T(n) = \frac{3n}{2} - 2 \in \Theta(n).$$

Scanned by CamScanner
Scanned by CamScanner

2 MARK QUESTIONS

1) Differentiate Divide and Conquer and Dynamic Programming.

Divide and Conquer	Dynamic Programming.
(i) It first divides the problem into small chunks or sub-problems.	(i) Here also the problem is divided into smaller sub-problems.
(ii) Here, sub-problems are solved recursively.	(ii) Here, the sub-problems are not solved recursively
(iii) It combines the result of sub-problems to arrive the final result.	(iii) Here, the key step is remembering, we store the result of sub-problem in a table format.
(iv) Sub-problems are independant of each other.	(iv) Sub-problems are not independant of each other.

2) Explain Divide and Conquer Strategy.

Divide and Conquer works in three steps,

Step 1: DIVIDE

It first divides the problem into smaller chunks or sub-problems.

Step 2: CONQUER

It then solve those sub-problems recursively

Scanned by CamScanner

Scanned by CamScanner

sub-problem.

Step:3 : COMBINE

It then combine the results of those sub-problems to arrive at a final result of the main problem.

Ex: Merge sort, Binary Search, Quick sort.

---

3) TIME COMPLEXITY OF ALL ALGORITHMS:

. BINARY SEARCH:

Worst - Case - Efficiency - $\theta(\log n)$

Best - Case - Efficiency - $\theta(1)$.

QUICK SORT

Worst - Case - Efficiency - $\theta(n^2)$

Best - Case - Efficiency - $\theta(n \log n)$

MERGE SORT

Worst - Case - Efficiency - $\theta(n \log n)$

Best - Case - Efficiency - $\theta(n \log n)$.

MINIMUM - MAXIMUM

Worst - Case - Efficiency - $\theta(n)$.

Best - Case - Efficiency - $\theta(n)$.

STRASSENS MATRIX MULTIPLICATION

Worst - Case - Efficiency - $\theta(n^{2.8})$.

Scanned by CamScanner

Scanned by CamScanner

4) What is - Stable Sorting. Is QuickSort, a Stable Sort?

STABLE SORTING:

* Stable Sorting Algorithms maintains the relative order of records with equal keys. That is a sorting algorithm is stable if whenever there are two records R and S with the same key and with R appearing before S in the original list, R will appear before S in the sorted list..

Ex: (10) (20) (20) (30) (5) [Before Sorting]

(5) (10) (20) (20) (30) [After Sorting]

Even after sorting orders of the 20's (Black and Blue) are maintained. Bubble Sort, Insertion Sort and MergeSo are examples of stable Sorting.

Is QuickSort a Stable Algorithm?

QuickSort is not a Stable Algorithm because it swaps non-adjacent elements.

Ex: 6 2_1 7_1 4 9 1 8 [5] 7_2 3_1 3_2 3_3 10 2_2 11 (⇒ Pivot)

After Applying QuickSort (After one iteration)

2_2 2_1 3_3 4 3_2 1 3_1 5 7_2 8 9 7_1 10 6 11

All the elements sharing the same key have had their order reversed.

Scanned by CamScanner

Scanned by CamScanner

5) Differentiate Quick Sort and Merge Sort.

BASIS FOR COMPARISON	QUICK SORT	MERGE SORT
) Partitioning of elements in the array.	The splitting of a list of elements is not necessarily divided into half.	Array is always divided into half (n/2).
i) Worst - Case Complexity	$O(n^2)$	$O(n \log n)$.
ii) Works well on	Smaller arrays	It works well on any type of array.
i) Speed	Faster than other sorting algorithms for small data set	Consistent speed in all types of data set.
i) Additional Space Requirement	Less	More
i) Efficiency	Inefficient for larger arrays	More Efficient
)Sorting Method	Internal	External.

Scanned by CamScanner

Scanned by CamScanner

6) Give the problem statement for sorting

Here, list of 'n' elements are given, must sort the list either in ascending or descending order.

I/p: Array A[0...n-1]

O/p: Sorted Array A.

---

7) Give the problem Statement for knapsack Problem.

Given a set of 'n' items, with known weights $w_1, w_2, \ldots w_n$ and values $v_1, v_2 \ldots v_n$ and a knapsack of capacity W. The aim of the problem is to find the most valuable subset of the items that fit into the knapsack.

Total possible subsets generated for n items is 2^n. Running time is $\theta(2^n)$. (Brute-Force

8) Give the problem statement for Travelling Salesman Problem (TSP)

Travelling Salesman Problem finds the shortest tour through a given set of n cities that visits each city exactly once and returns

Scanned by CamScanner

Scanned by CamScanner

to the city where it started.

TSP is modeled by a weighted graph with vertices representing cities, and edge weight representing the distance between the cities.

1) Give the problem statement for Assignment Problem.

There are n people who need to be assigned to execute n jobs, one person per job. Given a cost matrix C, where C[i, j] represents the cost that would acquire if i^{th} person is assigned to j^{th} job. The problem is to find an assignment with minimum total cost, i.e., Algorithm has to select a minimum value from each row 'n' such that

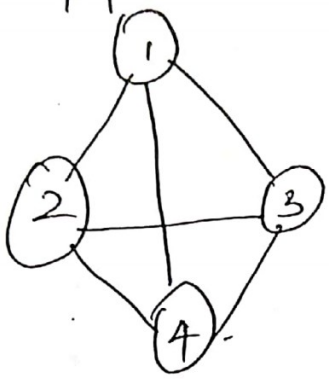
(i) selected values belong to different column
(ii) sum of selected values must be minimum.

Scanned by CamScanner
Scanned by CamScanner

10) Define Hamiltonian Circuit. Give Example.

Hamiltonian Circuit is defined as a cycle that passes through all the vertices of the graph exactly once, and comes back to th starting vertex. TSP is equivalent to the problem of finding the shortest hamiltonian circuit of the graph..

Graph



Hamiltonian Circuits.

1 - 2 - 4 - 3 - 1.

1 - 4 - 2 - 3 - 1

⋮

11) Define Brute Force Approach:

Brute force is a straightforward approach for solving a problem, usually directly based on the problem statement and definitions of the concepts involved. It is also called "Just Do It Approach".

Scanned by CamScanner

Scanned by CamScanner

12) Define Exhaustive Search.

Exhaustive Search is a Brute-force Approach to combinatorial problems. It generates each and every element of the problem's domain, selecting those of them that satisfy all the constraints and then finding the desired element.

13) What is Unstable Sorting.

* Unstable Sort is the converse of Stable Sort, where it is not necessary to maintain the relative order of records with equal key values. That is, a sorting Algorithm is unstable if whenever there are two records R and S with same key, R appears before S in the original list, but in the sorted list R may not be before S. Example: Quick Sort.

Scanned by CamScanner
Scanned by CamScanner

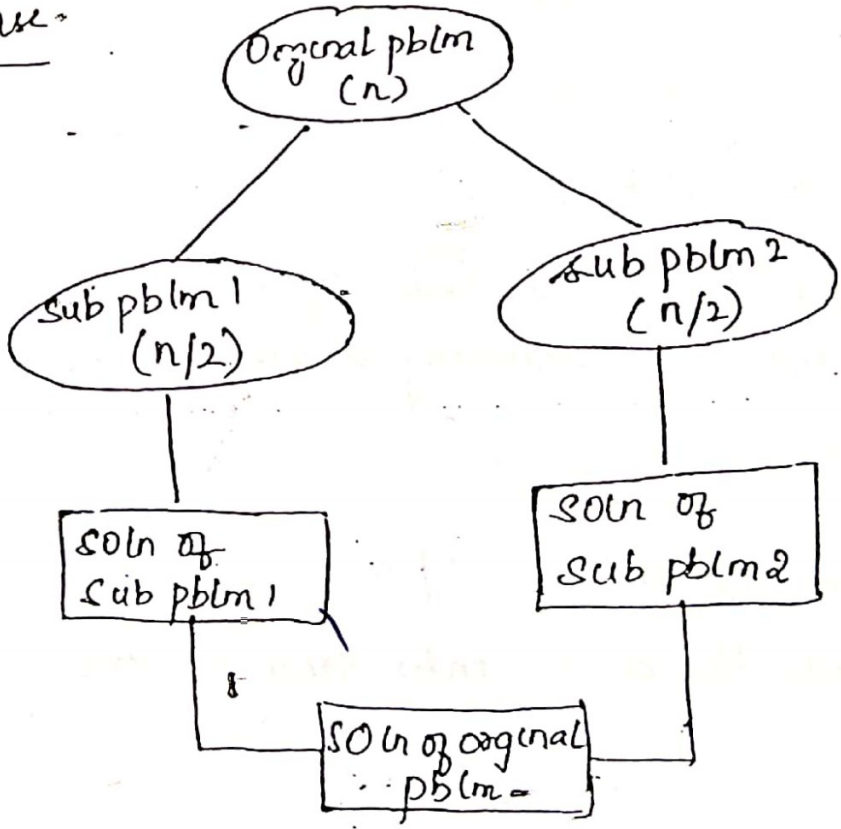
DIVIDE AND CONQUER TECHNIQUE

Divide and Conquer is the best-known algorithm design technique. It works according to the following general plan.

GENERAL-PLAN:

1. A problem's instance is divided into smaller instances of the same problem recursively, until it reaches a small enough to be solved directly.
2. The smaller instances are solved.
3. If necessary, the soln of the smaller instances are combined to get the soln for the orginal problem.

Eg: typical Case-



Scanned by CamScanner

Scanned by CamScanner

CONTROL ABSTRACTION :

```
DandC (P)
{
  if small(p) return soln(P);
  else
  {
    divide P into smaller instances P1, P2 .. Pk;
    Apply DandC to each smaller instance
    return Combine (DandC(P1), DandC(P2)
                    .... DandC(Pk));
  }
}
```

MERGE SORT :

* Merge sort algorithm is an perfect example of a successful application of the divide-and-conquer strategy.

* Problem Definition :

  Given with the list of 'n' elements, sort the list either in ascending or descending order.

* Solution :

  → Given an array A[0, ... n-1]

  → Divide the array into two halves

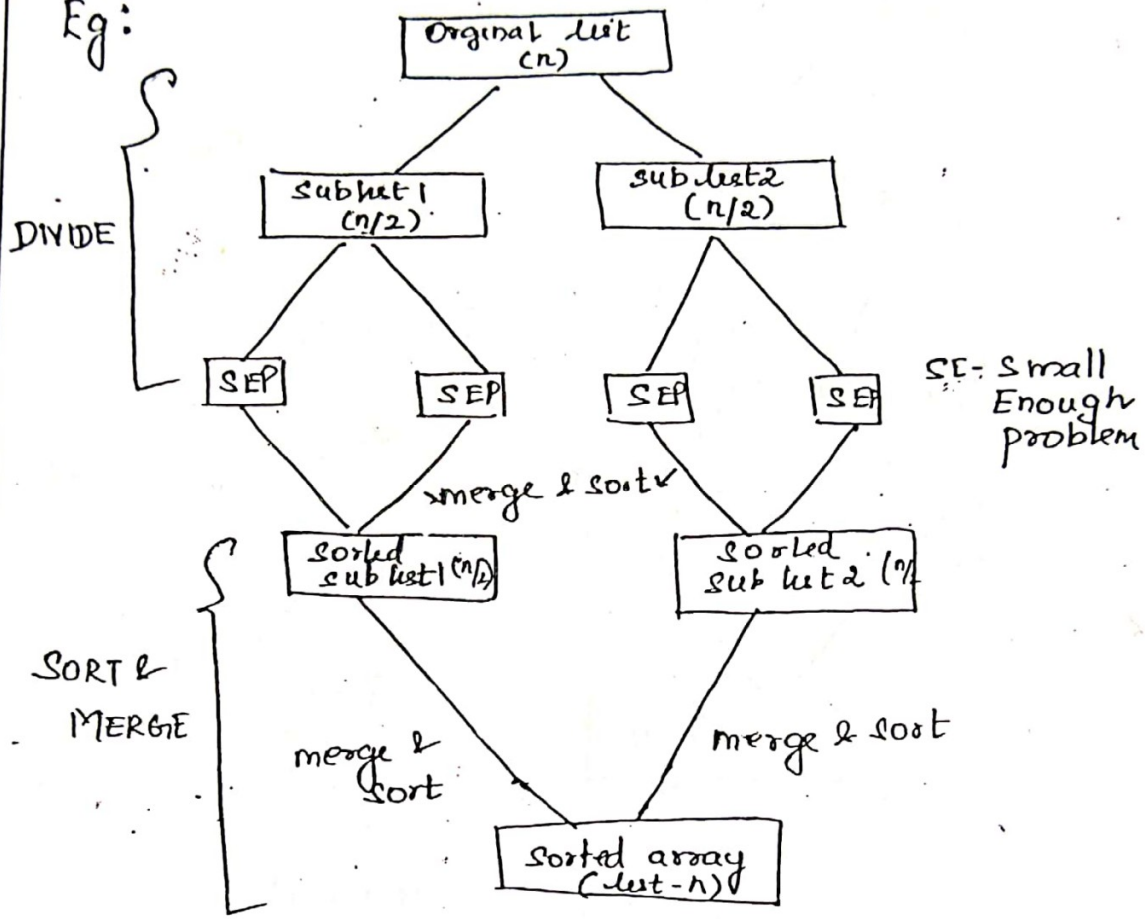
Scanned by CamScanner

Scanned by CamScanner

→ Divide until the problem becomes small enough and can be solved directly (recursively).

→ Then combine the soln of sub problems (sort at the same time) until the sorted array endsup as a single sorted array.

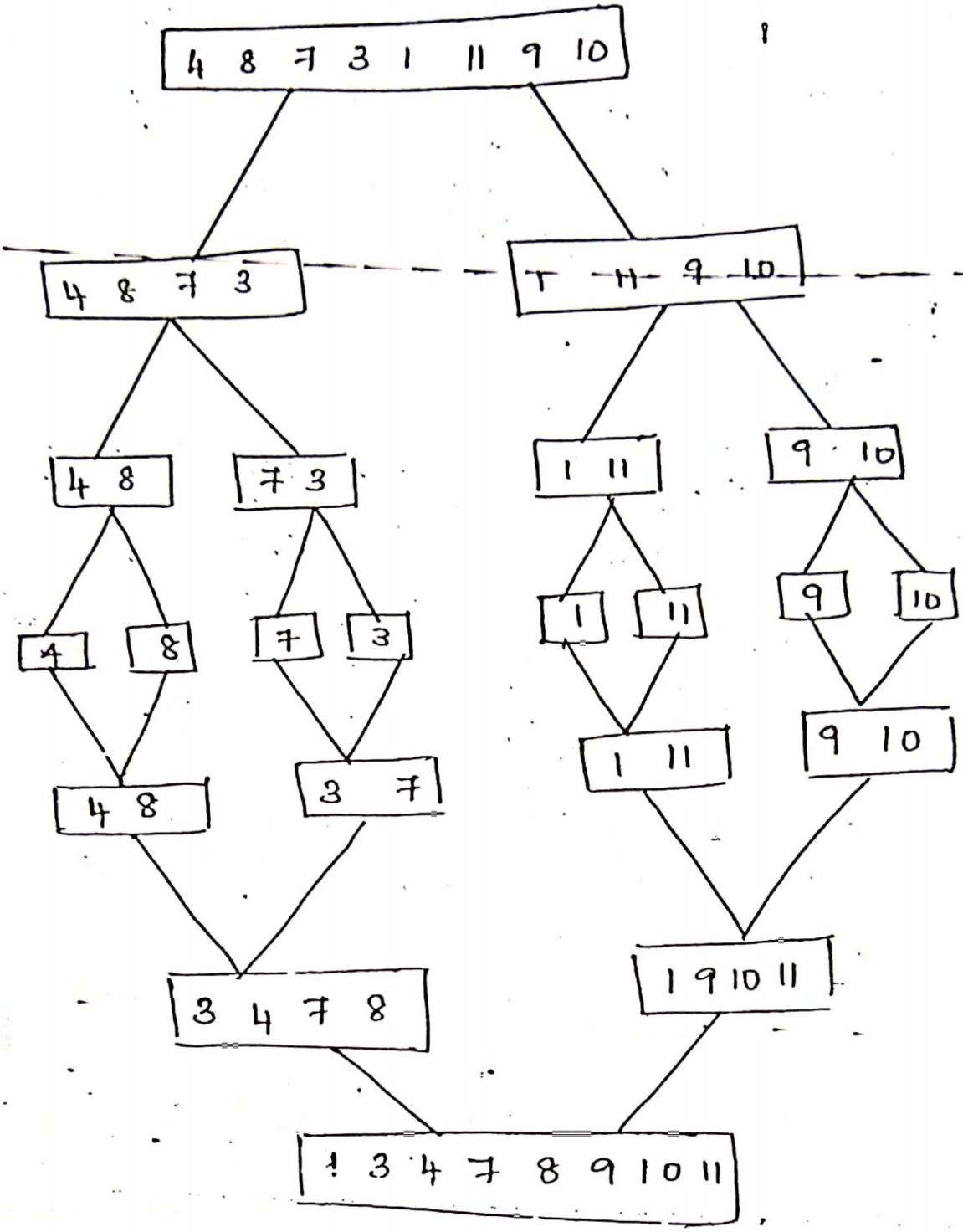
Eg:



Scanned by CamScanner

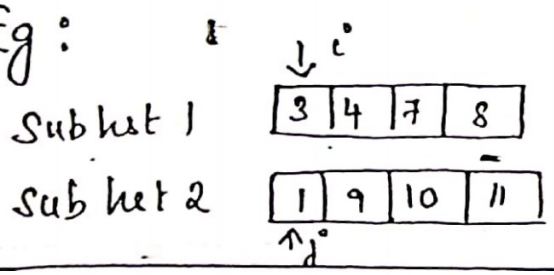
Scanned by CamScanner

Eg:

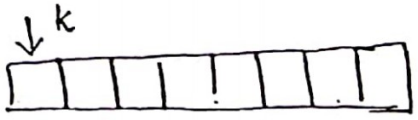


The following process or algorithm is employed to sort and merge to get the sorted list.

Eg:



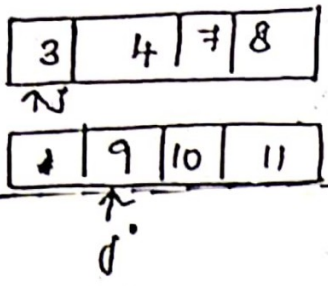
(empty list)

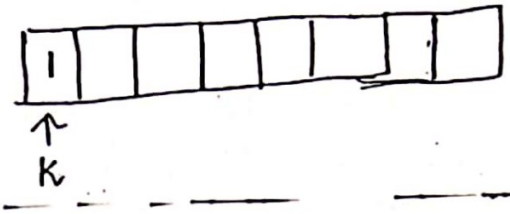


Scanned by CamScanner

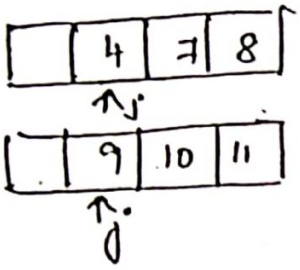
Scanned by CamScanner

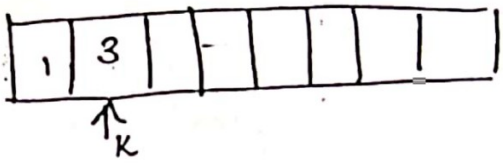
Compare i^{th} element (3) & j^{th} element (1), since '1' is small move it to the empty list and increment j^{th} pointer.

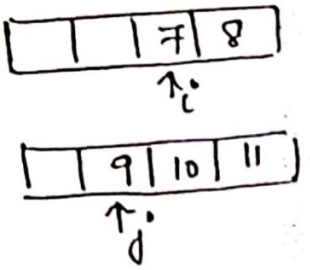


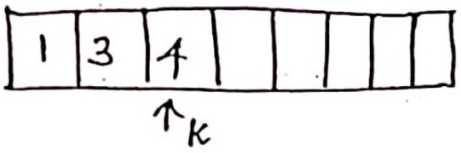


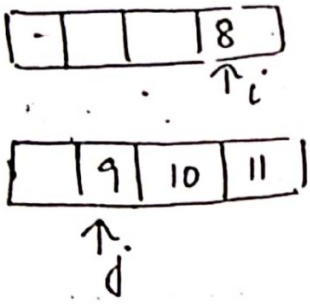
Continue the same process & increment the appropriate pointer

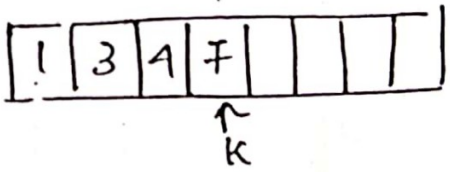


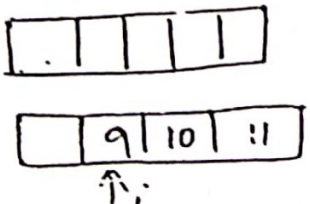


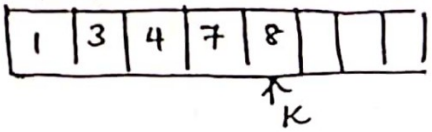


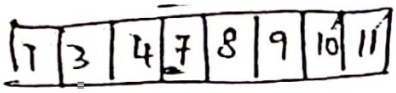












Since there is no elements in sublist 1, move all elements of sublist 2 to the sorted list.

Scanned by CamScanner

Scanned by CamScanner

Mergesort Algorithm :

```
Mergesort(A[0...n-1])
{
  //Input : Unordered Array A[0...n-1]
  //Output : Sorted Array A[0...n-1]
  if (n>1)
    {
      copy A[0...n/2-1] to B[0...n/2-1]
      copy A[n/2...n-1] to C[0...n/2-1]
      Mergesort(B[0...n/2-1])
      Mergesort(C[0...n/2-1])
      Merge(B,C,A)
    }
}

Merge(B[0...p-1], C[0,...q-1], A[0...p+q-1])
{
  // Merge two sorted array into one sorted array
  i←0 ; j←0 ; k←0
  While (i<p and j<p) do
    {
      if B[i]≤C[j]
        A[k]←B[i] ; i←i+1 ;
      else
        A[k]←C[j] ; j←j+1 ;
      k=k+1 ;
    }
  if(i==p)
    copy (C[j...q-1] to A[k...p+q-1])
  else copy (B[i...p-1] to A[k...p+q-1])
}
```

Scanned by CamScanner

Scanned by CamScanner

Analysis of Merge Sort:

→ For simplicity assume 'n' is in powers of 2. Hence the Recurrence Relation for the number of key comparison is,

$$C(n) = 2C(n/2) + C_{merge}(n), \text{ for } n > 1$$
$$C(1) = 0$$

→ First analyse the number of key comparisons performed during merging stage ($C_{merge}(n)$).

→ For that, in worst case, neither of the two arrays becomes empty before the other one just contains one element.

→ Therefore for the worst case, $C_{merge}(n) = n - 1$.

→ Now the Recurrence Relation becomes,

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1$$
$$C(1) = 0$$

→ Solving the recurrence using master theorem,

$a = 2$; $b = 2$; $d = 1$.

Case
- $a = b^d \Rightarrow 2 = 2^1$ ✓
- $a > b^d \Rightarrow \times$
- $a < b^d \Rightarrow \times$

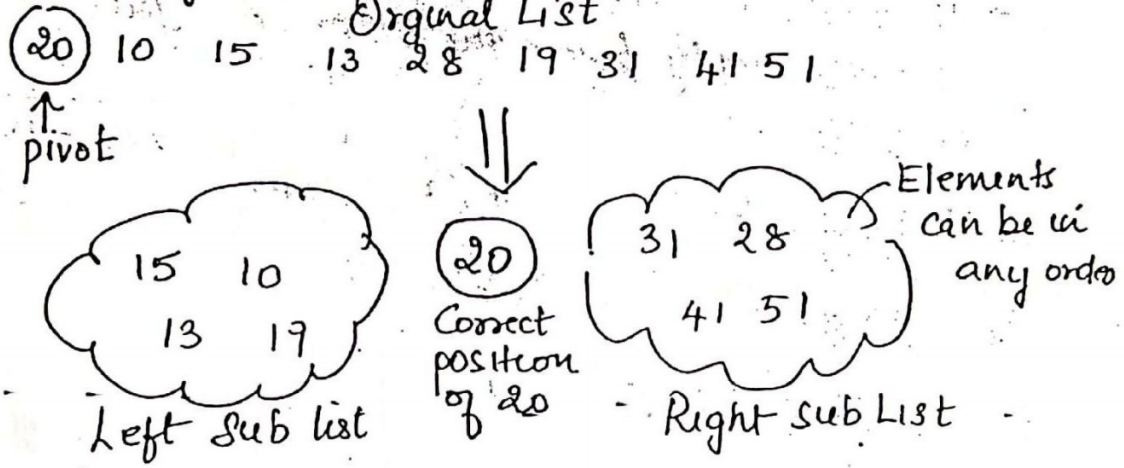
} Hence the Complexity is $\theta(n \log n)$

Scanned by CamScanner
Scanned by CamScanner

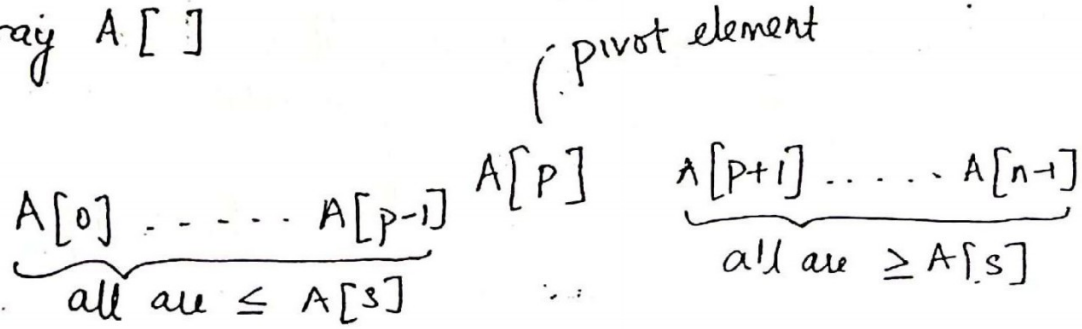
QUICK SORT : (SORTING ALGORITHM)

* Quick Sort is a sorting algorithm based on divide and conquer technique.
* It divides the list into two halves based on one element termed as pivot element.
* Based on a particular Strategy (Selecting the first element of the list as pivot), the pivot element is selected.
* The pivot element is brought to its correct position in the sorted list, by rearranging the elements in the list using a method.
* Now all the elements in the left of pivot will be smaller than pivot & all the elements in the right of pivot will be greater than it.

Eg:



* After dividing the list into two sub list based on the pivot element, the same method is applied to sort the sub lists recursively.



Scanned by CamScanner
Scanned by CamScanner

QUICK SORT ALGORITHM :

// Input :

```
Quicksort ( A[l...r] )
 {
   if l < r
     s ← partition (A[l...r])
     Quicksort (A[l...s-1])
     Quicksort (A[s+1...r])
 }

Partition (A[l..r])
 {
   p ← A[l]
   i ← l; j ← r+1
   repeat
      repeat i ← i+1 until A[i] ≥ p
      repeat j ← j-1 until A[j] ≤ p
      Swap (A[i], A[j])
   until i ≥ j
   Swap (A[i], A[j]) // undo last swap when i ≥ j
   Swap (A[l], A[j])
   return j
 }
```

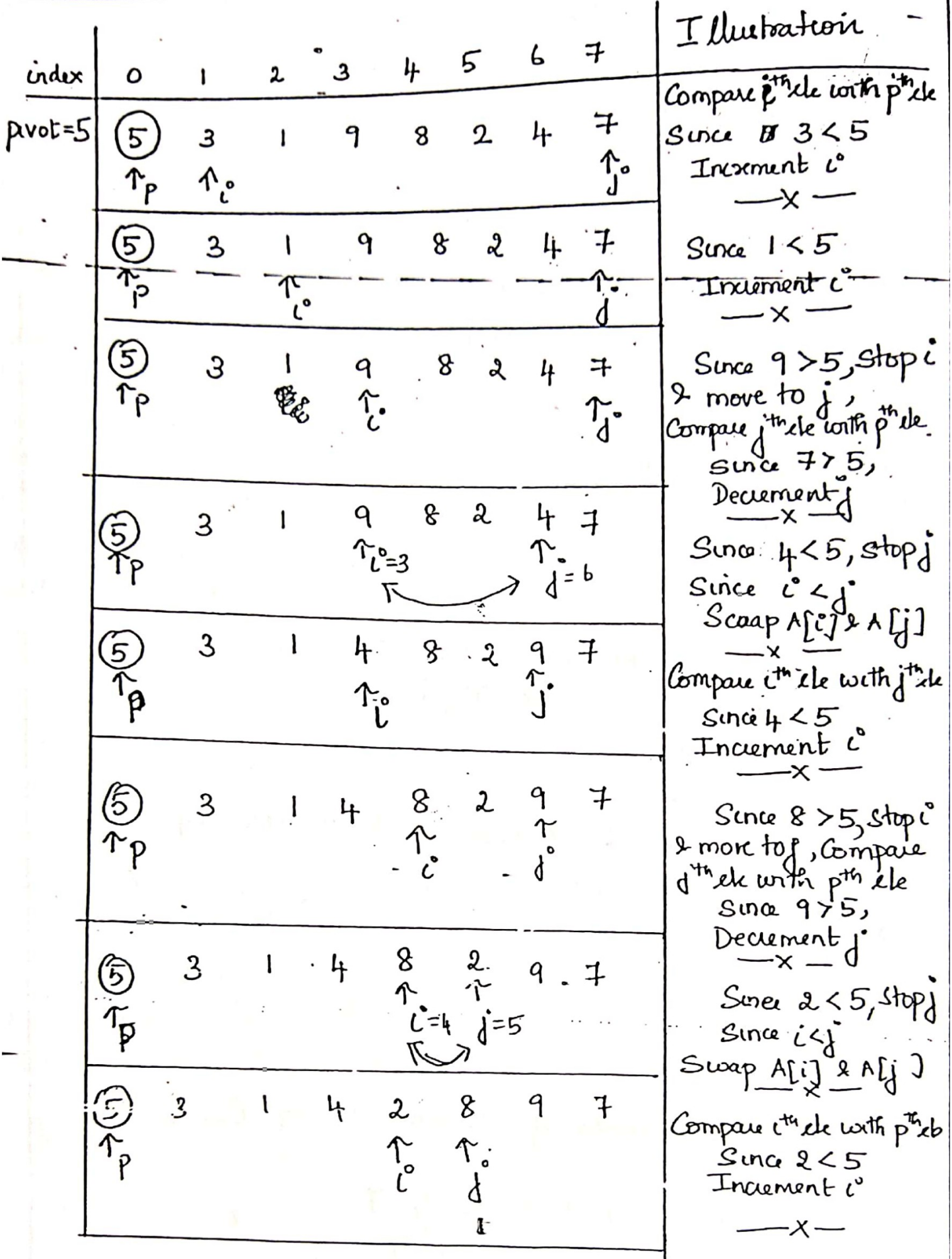
Example :

Sort the given list of elements using Quick Sort

5 3 1 9 8 2 4 7

Scanned by CamScanner

Scanned by CamScanner



Scanned by CamScanner

Scanned by CamScanner

Index	0	1	2	3	4	5	6	7	Illustration
	(5)	3	1	4	2	8	9	7	Since 8 > 5, Stop i° move to j°, Compare j^th ele with p^th ele Since 8 > 5, Decrement j
	↑p					↑ i,j			
	(5)	3	1	4	2	8	9	7	Since 2 < 5, Stop j Since i° > j° Swap A[P] & A[j]
	↑p				↑ j=4	↑ i=5			
	(2 3 1 4) (5) (8 9 7) Sub list 1 Sub List 2								Since pivot element is brought to its position in sorted List, list is divided into two.
	Sublist 1								
pivot=2	2	3	1	4					The Above same process is applied recursively to sort Sub list 1
	↑p	↑i		↑j					
	FINAL RESULT 1 2 3 4								
pivot=8	sublist 2								
	8	9	7						The Above same process is applied recursively to sort sub list 2
	↑p	↑i	↑j						
	FINAL RESULT 7 8 9								

Final Sorted list is

1 2 3 4 5 7 8 9

Analysis Of Quick Sort:

Step 1: Input Size - 'n' - no of elements

Step 2: Basic Operation - Comparison

Step 3: Executing time also depends on the arrangement Of the list (Instance), Find Best & Worst Case Efficiency.

Scanned by CamScanner

Scanned by CamScanner

Best Case Effecency:

For Best case, the partition always takes place in the middle of the list, hence the Reccurance relation is,

$$C_{Best}(n) = 2C_{Best}(n/2) + n \quad , n > 1$$

n → no of Comparison when partitioned in middle.

Solving the Recurrence Using Master theorem Method.

$a = 2 ; b = 2 ; d = 1$

$a = b^d \Rightarrow 2 = 2^1$ ✓ true hence,

$$C_{Best}(n) = \Theta(n^d \log n)$$

$$\boxed{C_{Best}(n) = \Theta(n \log n)}$$

Worst Case Effecency:

→ For Worst Case, the list is will be already sorted, (i.e) an already sort list is been sorted using quick sort leads to Worst Case.

→ In this the list will be partitioned, in such a way that, one sub list will be empty. The size of the other sublist will be decrement by 1.

→ Eg: No of Elements in the Orginal list = 5

Scanned by CamScanner

Scanned by CamScanner

Size of list/sublist	No of Comparisons (n+1)
Orginal List – 5	6
Sublist1 – 4	5
Sublist1.1 – 3	4
Sublist1.1.1 – 2	3

Total Comparison = 6+5+4+3 = 18

[One element no comparison]

[Only one sublist will have elements & another will be empty]

* Hence if there are 'n' elements in the list,

$$C_{(worst)}(n) = (n+1) + n + (n-1) + (n-2) + \cdots 3$$

$$= \frac{(n+1)(n+2)}{2} - 3$$

$$= \frac{n^2 + 2n + n + 2 - 6}{2}$$

$$= \frac{n^2 + 3n - 4}{2}$$

$$\boxed{C_{(worst)}(n) \approx \theta(n^2)}$$

Average Case Analysis:

Average Case Scenario: Assume that the partition happens in any position 's' of the array.

Analysis:

Assume that the partition at any position 's' happens with a probability of $1/n$. The recurrence relation is,

$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [n+1 + C_{avg}(s) + C_{avg}(n-1-s)], \quad n > 1$$

Solving this recurrence will end up in $\boxed{C_{avg}(n) \approx 1.38 n \log n}$

Scanned by CamScanner

Scanned by CamScanner